

```

1  REGFLOW v2.3 – SYSTEM ARCHITECTURE
2  =====
3
4
5  Regulatory Sources (EUR-Lex API, FCA Register, SEC EDGAR, FinCEN, Federal Register, ESMA)
6      |
7      ▼
8  Ingestion & Normalization Layer [No LLM – pure engineering]
9      - crawling / API ingestion
10     - document parsing (PDF / HTML / XML)
11     - metadata extraction (date, jurisdiction, document ID)
12     - article-level segmentation (from XML/HTML structural tags)
13     - version tracking / redlining
14     - WRITES TO: raw document store (originals) + vector store (embedded chunks)
15         |
16         ▼
17     [
18     |  EVENT SOURCE (not a pipeline stage)
19     |
20     |  Agent 1: Regulatory Radar
21     |      - rule-based: deterministic change detection
22     |          (diff against prior version)
23     |      - LLM-assisted: topic and severity classification
24     |          (cosmetic vs substantive)
25     |      - jurisdiction and regulator tagging
26     |      - confidence score attached
27     |      - FILTERS OUT: cosmetic changes, formatting-only diffs
28     |      - EMITS EVENT: RegulatoryChangeEvent
29     |          { article_id, severity, jurisdiction, regulator,
30     |              confidence, diff_summary }
31     |
32     |
33     |      ▼ (event triggers pipeline only for substantive changes)
34     |
35  Agent Orchestrator (LangGraph Workflow Engine) [CONTROL PLANE]
36      - subscribes to RegulatoryChangeEvent stream
37      - controls agent sequencing
38      - manages fan-out / fan-in around Agent 2 output (see below)
39      - applies per-agent confidence thresholds
40      - routes: AUTO / NOTIFY / BLOCK based on confidence + severity
41      - triggers human review when thresholds breached
42      - manages retries & error recovery
43      - tracks workflow status & scheduling
44          |
45          ▼
46  Agent 2: Obligation Extractor [FAN-OUT POINT]
47      - receives one article/section at a time
48      - outputs zero, one, or multiple structured obligation JSON objects

```

- 49 - extracts: deadlines, scope, penalties, exemptions
- 50 - attaches source citations + confidence score
- 51 - READS FROM: vector store (RAG over legal corpus) +
- 52 override store (top-k few-shot corrections,
- 53 see feedback loop)
- 54 - WRITES TO: knowledge graph (obligation nodes + edges) +
- 55 vector store (re-embeds new obligations for downstream
- 56 semantic search)
- 57 - ▲ low confidence → mandatory human review

58 |

59 ▼

```
60 |-----|
61 | ORCHESTRATOR FAN-OUT / FAN-IN CONTRACT |
62 |-----|
63 | If Agent 2 emits N obligations from one article: |
64 |-----|
65 | Agent 3 (Conflict Detector): |
66 |   - BATCH MODE – receives all N obligations as a set |
67 |   - runs once per article, not N times |
68 |   - set operation against knowledge graph + vector store |
69 |-----|
70 | Agents 4, 5, 6: |
71 |   - PARALLEL FAN-OUT – one instance per obligation |
72 |   - fan-in at Human Review Layer (aggregated per article) |
73 |-----|
74 | Empty output (N=0): pipeline terminates, logs no-op |
75 |-----|
```

76 |

77 ▼

- 78 Agent 3: Cross-Jurisdiction Conflict Detector [BATCH]
- 79 - READS FROM: knowledge graph + vector store + enterprise context
 - 80 + raw source citations + override store
 - 81 - vector store used for semantic discovery of unlinked conflicts
 - 82 - override store used for false-positive correction retrieval
 - 83 - detects contradictions (e.g., retention 5yr vs 7yr)
 - 84 - detects overlaps (duplicative obligations across EU/UK/US rules)
 - 85 - detects stricter-standard cases (one jurisdiction exceeds another)
 - 86 - ▲ high-severity conflict → human legal signoff

87 |

88 ▼

- 89 Agent 4: Gap Analyzer & Risk Scorer [PARALLEL per obligation]
- 90 - READS FROM: knowledge graph + enterprise context + override store
 - 91 - maps obligations to existing controls
 - 92 - identifies missing / weak controls
 - 93 - checks related prior audit findings
 - 94 - checks whether evidence already exists
 - 95 - penalizes recurring unresolved gaps
 - 96 - multi-factor risk score:

```
97         enforcement severity × business impact × deadline urgency
98     - classifies: HIGH / MEDIUM / LOW
99     - ▲ high-risk gap or remediation plan → compliance approval
100         |
101         ▼
102 Agent 5: Remediation Planner [PARALLEL per obligation]
103     - READS FROM: enterprise context + override store
104     - action plan per gap
105     - SUGGESTS owners & deadlines (reads control owners / responsible teams)
106     - incorporates past reviewer overrides for owner suggestions
107     - proposes control / policy updates
108     - dependency-aware prioritization
109         |
110         ▼
111 Agent 6: Audit Evidence Generator [PARALLEL per obligation]
112     - READS FROM: knowledge graph + raw document store + override store
113     - traceable evidence pack
114     - clause citations + control links
115     - evidence-backed justification
116     - review log
117         |
118         ▼ (fan-in: aggregate all N obligation outputs per article)
119         |
120 Human Review & Approval Layer
121     - legal validation
122     - compliance team approval
123     - audit export workflow
124     - confidence-triggered reviews:
125         • low-confidence extraction → mandatory review
126         • high-severity conflict → legal signoff
127         • high-risk gap or remediation plan → compliance approval
128     - WRITES TO: review log / override store (structured correction records)
129
130
131 FEEDBACK LOOP MECHANISM [concrete, not aspirational]
132 =====
133
134 Each reviewer correction is stored as a structured record in the
135 Override Store, which maintains its OWN embedding index (separate
136 from the regulatory vector store):
137
138 {
139     agent_id,                // which agent was corrected
140     input_context,           // original article/obligation text
141     original_output,         // what the agent produced
142     corrected_output,        // what the reviewer changed it to
143     correction_type,         // wrong_extraction | false_positive_conflict
144                             // | risk_override | owner_reassignment
```

```
145     embedding                // vector embedding of input_context
146 }
147
148 At inference time, each agent performs:
149     1. Embed current input (article / obligation / conflict candidate)
150     2. Retrieve top-k correction records from the Override Store
151        by cosine similarity, filtered by correction_type matching
152        that agent
153     3. Inject retrieved corrections into prompt as few-shot examples
154     4. Agent reasons with corrections as in-context guidance
155
156 No fine-tuning. No prompt rewrites. No model retraining.
157 Pure retrieval-augmented correction – corrections compound over time
158 without changing any model weights.
159
160 Why the Override Store is a separate indexed store (not part of the
161 regulatory vector store):
162     - different write pattern (reviewer decisions vs corpus ingestion)
163     - different retention policy (audit-grade, append-only)
164     - different access control (reviewer-attributable data)
165     - different embedding target (input_context, not legal clause)
166     - different retrieval scope (corrections only, per agent_id filter)
167
168 This is the system's primary novelty and measurable contribution:
169 correction-retrieval-augmented agents with empirical convergence
170 curves per agent (error rate vs. correction corpus size).
171
172
173 CONFIDENCE SCORING [honest framing]
174 =====
175
176 MVP (v2.3):
177     All confidence scores are LLM self-reported scalar values on a
178     0.0-1.0 scale, elicited via structured output
179     (JSON schema field: confidence: float).
180
181 Known limitations of self-reported confidence:
182     - poorly calibrated in practice (models sound confident when wrong)
183     - no independent verification signal
184     - routing critical review decisions off self-reported confidence
185       alone is risky at production scale
186
187 Roadmap for production-grade calibration:
188     - logprob-derived confidence for structured-output tokens
189     - secondary verifier model (smaller, task-specific) for
190       high-severity routing decisions
191     - empirical threshold calibration per agent, learned from the
192       review log (target: reviewer agreement rate at threshold)
```

193 - ensemble agreement across multiple sampled generations
194 for high-stakes obligations
195
196 The MVP is explicit about this ceiling. Calibration is a v3
197 concern, not a v2 claim.
198
199
200 SHARED DATA LAYERS (persistent infrastructure, read by multiple agents)
201

202
203 Neo4j Knowledge Graph
204 - structured relationships (simplified):
205 regulation → obligation → control → evidence
206 - full schema includes: regulation, document, clause, obligation,
207 control, policy, evidence, owner, jurisdiction, regulator
208 - written by: Agent 2
209 - read by: Agents 3, 4, 5, 6
210
211 Vector Store (Chroma / Weaviate) – REGULATORY CORPUS
212 - semantic search over regulatory corpus for RAG +
213 conflict discovery of unlinked obligations +
214 newly extracted obligations (re-embedded by Agent 2)
215 - written by: Ingestion Layer, Agent 2
216 - read by: Agents 2, 3
217
218 Raw Document Store (PostgreSQL / S3)
219 - original text preserved for legal traceability & source citations
220 - written by: Ingestion Layer
221 - read by: Agents 3, 6
222
223 Enterprise Context Layer [Manual Upload – Approach A]
224 - internal controls
225 - policies / SOPs
226 - risk register
227 - business units
228 - products / entities / vendors
229 - prior audit findings
230 - control owners / process owners / responsible teams
231 - read by: Agents 3, 4, 5
232
233 Override Store – CORRECTION RECORDS (with its own embedding index)
234 - reviewer decisions
235 - corrected outputs
236 - false positive labels
237 - approved mappings
238 - score overrides
239 - embeddings of input_context (for retrieval-augmented correction)
240 - separate from regulatory vector store – different schema,

